Motore grafico e viewer 3D

Progetto software in UML (Series 4), versione completa e spiegata · Computer Graphics M-I5203, SUPSI · Alessio Gervasini · ottobre 2026

> **Milestone Series 4.** Il codice degli esercizi diventa una libreria con un'API: classi per mesh, luci, materiali, camere. L'applicazione usa solo queste classi e non chiama mai OpenGL o FreeGLUT. Questo documento mostra il progetto con cinque diagrammi UML e, per ognuno, spiega **ogni riquadro e ogni freccia**: che tipo di freccia è, cosa significa, cosa una classe eredita dall'altra, chi crea e chi possiede ogni oggetto e in quale riga del codice succede. Non serve conoscere il progetto: la sezione 0 introduce tutte le parole e i simboli usati dopo.

> **Come leggere i riferimenti al codice.** Una scritta come `node.cpp:59–63` indica il file `engine/node.cpp`, righe 59–63. `main.cpp` è sempre `client/main.cpp`; tutti gli altri file sono in `engine/`. Ogni affermazione del documento si può verificare aprendo quella riga.

0. Prima di iniziare

0.1 Il progetto in poche righe

Il progetto è formato da **due programmi** che lavorano insieme:

- **il motore grafico** (`engine.dll`): una libreria, cioè codice che non si avvia da solo ma offre delle classi pronte (camera, luce, mesh, …). Sa aprire una finestra, leggere un modello 3D da file e disegnarlo con luci, texture e ombre;
- **il viewer** (`client.exe`): un programma piccolo (234 righe) che usa il motore per mostrare una camera da letto in 3D. L'utente si muove con `W A S D` e il mouse, cambia camera con `C`, accende e spegne la torcia (`1`) e la lampada (`2`).

Separare i due programmi è il punto della milestone: il motore si potrà riusare per altre applicazioni (per esempio il gioco finale), e chi scrive il viewer non deve sapere niente di OpenGL.

0.2 Parole da conoscere

Parola	Significato nel progetto
OpenGL	La libreria che parla con la scheda video e disegna i triangoli. Usiamo la versione 1.x a “pipeline fissa” (luci e materiali già pronti, niente shader), come richiesto dal corso.
FreeGLUT	Libreria che apre la finestra, legge tastiera e mouse e scambia i buffer dell'immagine.
Assimp	Libreria che legge i file 3D (nel nostro caso `.glb`) e li trasforma in strutture C++.
GLM	Libreria matematica: vettori (`glm::vec3`) e matrici 4×4 (`glm::mat4`). È fatta solo di header: non c'è una `.dll` da collegare.
DLL / API	DLL = libreria caricata dal programma all'avvio. API = l'insieme delle classi e funzioni che la libreria rende pubbliche (qui tutto ciò che sta nel namespace `Eng::`).
Mesh	Un oggetto 3D fatto di triangoli. Ogni vertice ha una posizione, una **normale** (la direzione verso cui “guarda” la superficie, serve per la luce) e due coordinate **UV** (quale punto dell'immagine-texture va su quel vertice).
Materiale / texture	Materiale = come la superficie reagisce alla luce (colori ambiente, diffuso, speculare, emissione, lucidità). Texture = immagine incollata sulla superficie.

Luce direzionale / puntiforme	Direzionale: raggi paralleli, come il sole; conta solo la direzione. Puntiforme: parte da un punto, come una lampadina, e si indebolisce con la distanza.
Matrice 4×4	Una tabella di 16 numeri che dice dove sta un oggetto e come è ruotato e scalato. Moltiplicare due matrici = applicare una trasformazione dopo l'altra.
Scene graph	Albero di oggetti: ogni nodo ha una matrice *relativa al padre*. Se sposti il padre, si spostano anche tutti i figli (come spostare un tavolo con sopra una lampada).
Display list	Comandi OpenGL registrati una volta sola e poi ripetuti con un solo `glCallList`: disegnare una mesh diventa velocissimo.
Stencil buffer	Un'immagine nascosta grande come la finestra, con un numero intero per pixel. Serve a "ritagliare": qui segna i pixel del pavimento e quelli in ombra.
Classe astratta	Classe di cui non si possono creare oggetti: serve solo come base per altre classi.
Metodo virtuale / override	Un metodo che le classi figlie possono riscrivere. Quando il programma chiama `render()` su un puntatore generico, parte la versione della classe vera dell'oggetto (**polimorfismo**).
Singleton	Classe di cui esiste un solo oggetto in tutto il programma (qui: una sola finestra).
Puntatore / shared_ptr	Puntatore = indirizzo di un oggetto. `std::shared_ptr` = puntatore "condiviso": l'oggetto viene cancellato da solo quando l'ultimo che lo usa lo lascia.

0.3 Come si leggono le frecce UML

UML è un modo standard di disegnare un programma. Ogni tipo di freccia ha un significato preciso: la forma della **punta** e il tipo di **linea** (continua o tratteggiata) dicono che rapporto c'è tra i due riquadri.

Simbolo	Nome	Si legge	Dove compare qui
	**Generalizzazione** (ereditarietà)	"A **è un** B". Il triangolo vuoto punta alla classe madre. La figlia riceve tutto quello che ha la madre (dati e metodi) e può aggiungere o riscrivere.	Diagramma 2: 9 frecce (es. Mesh → Node)
	**Composizione**	"A **possiede** B". Il rombo pieno sta dal lato del proprietario. Quando il proprietario viene distrutto, distrugge anche le parti.	Diagramma 2: Node → Node (figli); diagramma 4: tutte le frecce
	**Aggregazione**	"A **usa** B, ma non ne è l'unico padrone". Rombo vuoto dal lato di A. B può essere condiviso e sopravvivere ad A.	Diagramma 2: Mesh → Material, Material → Texture, List → Node
	**Dipendenza**	"A **ha bisogno** di B per lavorare", ma non lo tiene come dato: lo riceve come parametro, lo crea e lo restituisce, oppure usa solo i suoi tipi.	Diagramma 1: verso GLM; diagramma 2: Loader → Node, Base → List
	**Associazione** (uso diretto)	"A **comunica con / usa** B" in modo stabile. Nel diagramma 1 indica una libreria collegata e chiamata mentre il programma gira; nel diagramma 5 indica "diventa".	Diagrammi 1 e 5
	**Messaggio** (diagramma di sequenza)	"A **chiama** un metodo di B". Una freccia tratteggiata in un diagramma di sequenza è invece il **valore restituito**.	Diagramma 3

Numeri sulle frecce (molteplicità). `0..*` vicino a un estremo significa "da zero a quanti se ne vuole". Se non c'è un numero, il rapporto è con un solo oggetto (o con nessuno, se il puntatore può essere vuoto).

Simboli dentro i riquadri delle classi (diagramma 2):

- **C** nel cerchio = classe normale; **A** nel cerchio e nome in *corsivo* = classe astratta;
- `«singleton»` sopra il nome = **stereotipo**, un'etichetta che dice “di questa classe esiste un solo oggetto”;
- `+` davanti a un metodo = pubblico (lo può chiamare chiunque);
- metodo in *corsivo* = astratto (le figlie **devono** scriverlo); metodo sottolineato = statico (si chiama sulla classe, senza creare un oggetto: `Loader::load(...)`);
- le classi senza righe di metodi (Mesh, Camera, …) non sono vuote: il diagramma mostra solo i metodi più importanti per restare leggibile. L'elenco completo è nella sezione 2.2.

Diagramma degli oggetti (diagramma 4): ogni riquadro è un *oggetto vero* che esiste mentre il programma gira, scritto `nome : Classe`. **Diagramma di sequenza** (diagramma 3): il tempo scorre dall'alto verso il basso; ogni colonna è un partecipante, la linea tratteggiata verticale è la sua “linea della vita”.

1. Architettura: chi usa cosa

Il primo diagramma è il più generale: non mostra classi ma **pezzi di software** (programmi, librerie, file) e chi dipende da chi.

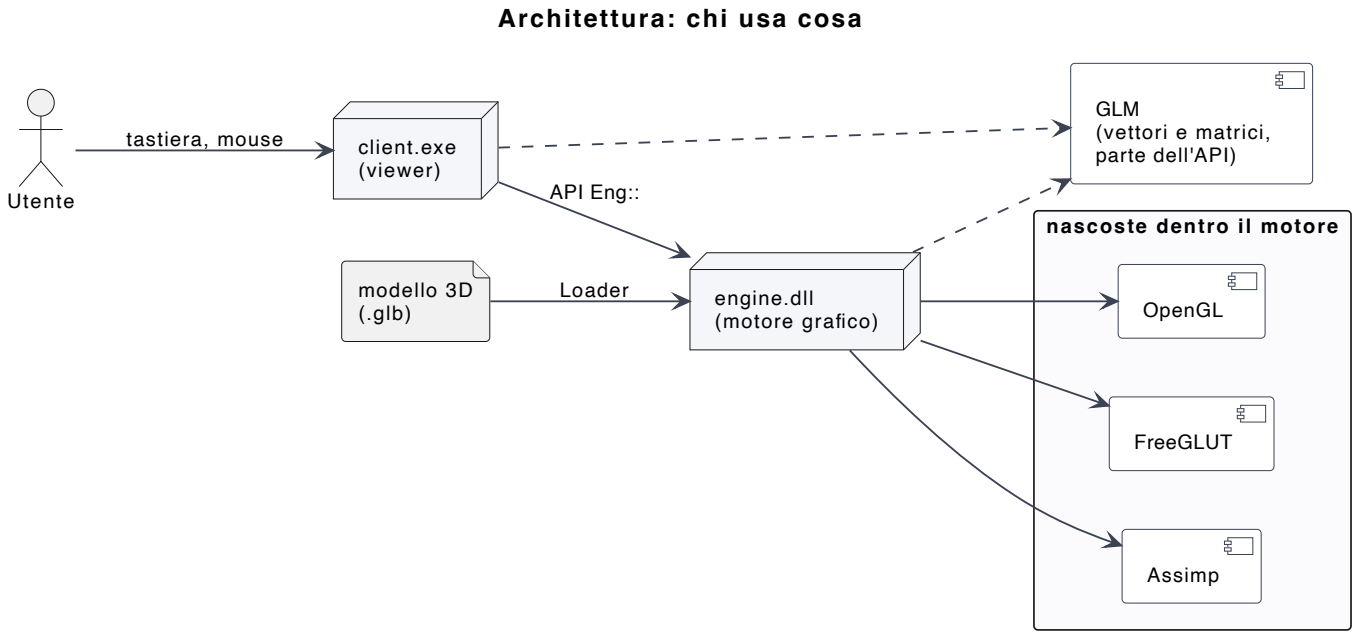


Figura 1 – Componenti e dipendenze.

1.1 Gli elementi

Elemento	Simbolo UML	Cos'è e cosa fa
Utente	attore (omino)	Chi usa il programma: preme i tasti e muove il mouse. Un attore sta sempre fuori dal software.
modello 3D (.glb)	file (foglio con l'angolo piegato)	Il file `client/assets/bedroom_furniture_set/bedroom_furniture_set_-_game_ready.glb` con mobili, materiali, texture e la luce della lampada. Il percorso è scritto in `main.cpp:14`.
client.exe (viewer)	nodo (scatola 3D) = programma eseguibile	Il programma che si avvia. Tutto il suo codice è in `client/main.cpp`.
engine.dll (motore grafico)	nodo	La libreria. Header pubblici in `engine/*.h`, implementazione in `engine/*.cpp` (12 classi, circa 1 960 righe).
GLM	componente	Matematica. Unica libreria esterna che il viewer vede, perché l'API del motore usa `glm::vec3` e `glm::mat4` nei parametri (Series 1). Inclusa da `engine.h:14–18`.
OpenGL, FreeGLUT, Assimp	componenti dentro il rettangolo "nascoste dentro il motore"	Le tre librerie che fanno il lavoro vero. Il rettangolo è un raggruppamento: dice che il viewer non le vede (vedi 1.3).

1.2 Le frecce una per una

#	Freccia	Tipo	Cosa significa	Nel codice

1	Utente → client.exe *“tastiera, mouse”*	continua	L'utente comanda il viewer. Dal suo punto di vista parla con il viewer; tecnicamente la finestra appartiene al motore: FreeGLUT riceve il tasto, il motore lo passa al viewer chiamando la funzione registrata dal viewer (*callback*), e il viewer chiede a ogni frame quali tasti sono premuti e quanto si è mosso il mouse.	registrazione: `main.cpp:189`; tasti a evento: `main.cpp:112–141` (C, 1, 2, F, P); tasti tenuti premuti e mouse: `main.cpp:147–173`; lato motore: `engine.cpp:162–206`
2	client.exe → engine.dll *“API Eng::”*	continua	Il viewer usa le classi del motore. Include un solo header, `engine.h`, e scrive soltanto codice `Eng::...`: `Base`, `Loader`, `Node`, `Camera`, `DirectionalLight`, `List`. Su Windows la macro `ENG_API` diventa `__declspec(dllexport)` quando si compila la DLL e `__declspec(dllimport)` quando la usa il client: è ciò che rende le classi visibili fuori dalla DLL.	include: `main.cpp:8`; uso: `main.cpp:186–236`; macro: `engine.h:26–36`
3	client.exe ⇢ GLM	tratteggiata (dipendenza)	Il viewer usa i tipi e le funzioni di GLM per costruire le matrici delle camere. Tratteggiata perché GLM è solo header: non c'è niente da collegare quando il programma parte.	costanti `main.cpp:20–22`; `glm::lookAt` `main.cpp:104`; `glm::rotate/translate` `main.cpp:158–172`
4	modello .glb → engine.dll *“Loader”*	continua	Il file entra nel motore solo attraverso la classe `Loader`. L'etichetta dice *quale* parte del motore lo legge. Il viewer passa solo il nome del file.	`main.cpp:193` → `loader.cpp:178–194`
5	engine.dll → OpenGL	continua	Il motore disegna chiamando le funzioni `gl...`: matrici, luci, materiali, texture, display list, stencil.	`mesh.cpp:53–69`, `light.cpp:65–76`, `material.cpp:97–109`, `texture.cpp:46–51`, `list.cpp:62–128`, `engine.cpp:286–292`
6	engine.dll → FreeGLUT	continua	Il motore apre la finestra, registra le funzioni che ricevono tastiera e mouse, misura il tempo, scambia i buffer.	`engine.cpp:266–280` (apertura), `engine.cpp:456–475` (scambio buffer e eventi)
7	engine.dll → Assimp	continua	Il `Loader` fa leggere il file ad Assimp e poi converte il risultato nelle classi del motore.	include `loader.cpp:12–14`; lettura `loader.cpp:180–181`
8	engine.dll ⇢ GLM	tratteggiata (dipendenza)	Anche il motore fa i suoi calcoli con GLM: matrice mondo, matrice di vista, proiezione, matrice dell'ombra.	`node.cpp:48–53`, `camera.cpp:31–44`, `list.cpp:98–100`

1.3 Perché OpenGL, FreeGLUT e Assimp sono “nascoste”

- Gli header pubblici (`engine/*.h`) includono solo la libreria standard e GLM (`engine.h:9–18`): nessuna riga nomina OpenGL o FreeGLUT. La Series 4 chiede proprio questo: chi usa la libreria non deve nemmeno sospettare cosa c'è sotto.
- OpenGL e FreeGLUT sono incluse solo nei file `.cpp` (`#include <GL/freeglut.h>` in `engine.cpp:15`, `mesh.cpp:10`, `light.cpp:10`, `pointlight.cpp:10`, `material.cpp:10`, `texture.cpp:10`, `list.cpp:10`); Assimp solo in `loader.cpp:12–14`.
- I dati interni di `Base` legati a FreeGLUT (id della finestra, tasti premuti, tempi) stanno in una struttura `Reserved` dichiarata nell'header ma definita solo nel `.cpp` (`engine.h:92–93`, `engine.cpp:126–215`). Questa tecnica si chiama **PIMPL** (“pointer to implementation”).

- Le classi che contengono numeri di OpenGL li tengono come `unsigned int` o `int` (`Mesh::displayList`, `Texture::textureId`, `Light::lightId`), non come tipi `GLuint`/`GLenum`, così l'header non ha bisogno di OpenGL.
- Effetto pratico: si potrebbe sostituire FreeGLUT o Assimp senza cambiare una riga del viewer.

2. Classi del motore

Il diagramma più importante: le 12 classi del namespace `Eng` e i loro rapporti.

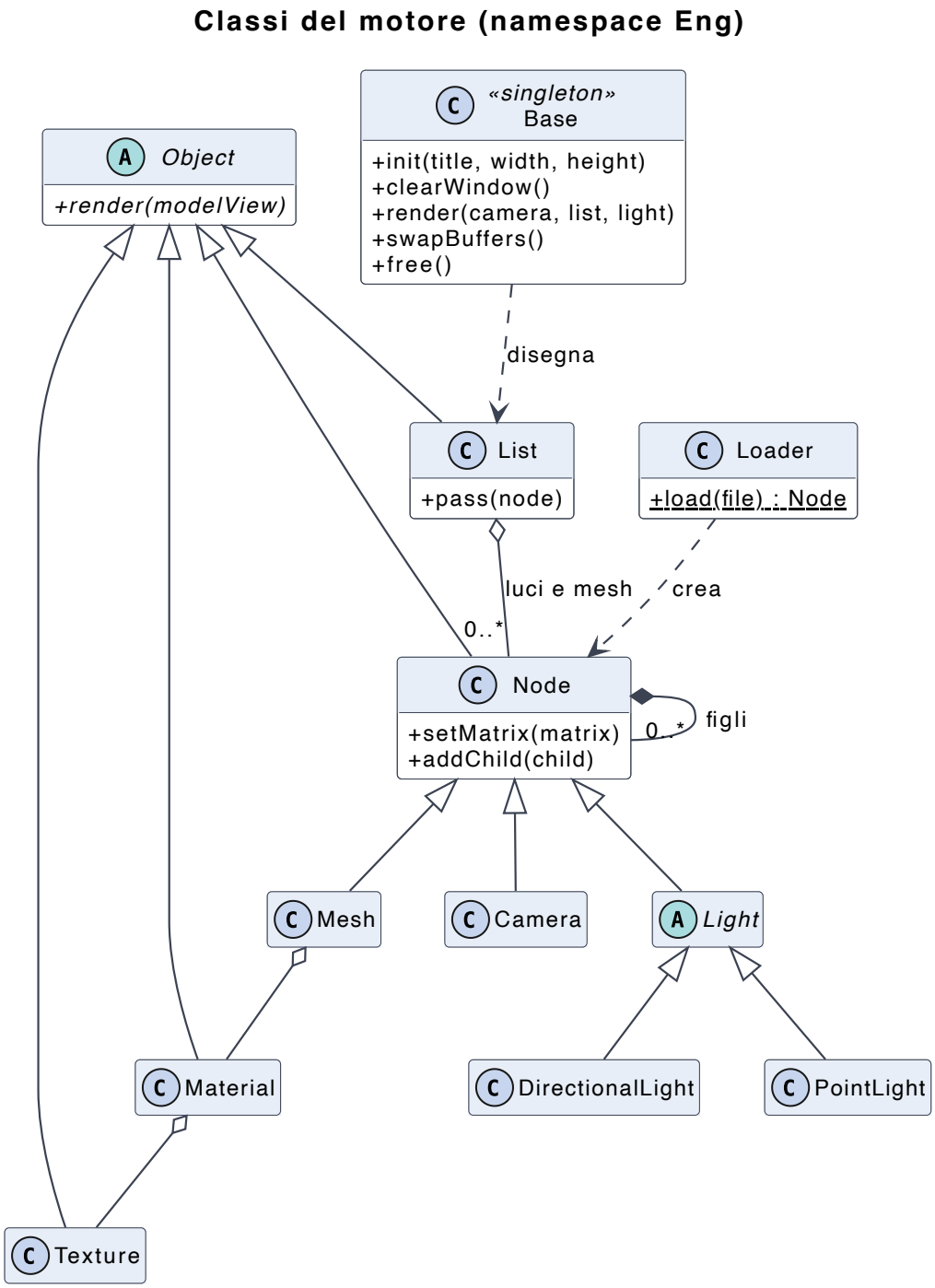


Figura 2 – Classi del motore e loro relazioni.

2.1 La struttura in una frase

Quasi tutto discende da **Object** (“qualcosa che si può disegnare”). Da Object nascono **Node** (le cose che stanno *in un punto della scena*) e tre classi che non hanno una posizione: **Material**, **Texture** e **List**. Da Node nascono le cose concrete della scena: **Mesh**, **Camera** e **Light**, che a sua volta si divide in **DirectionalLight** e **PointLight**. Restano fuori dalla gerarchia **Base** (la finestra) e **Loader** (il lettore di file), che non sono oggetti della scena ma “servizi”.

2.2 Le 12 classi, una per una

Base «singleton» — `engine.h:56–96`, `engine.cpp`

Eredita da	nessuno. È `final` (nessuno può ereditare da lei).
Perché singleton	Costruttore privato (`engine.h:95`), copia vietata (`engine.h:65, 68`): l'unico modo di averla è `Base::getInstance()`, che restituisce sempre lo stesso oggetto statico (`engine.cpp:241–245`). Una finestra e un contesto OpenGL per programma.
Metodi nel diagramma	`init(title, width, height)` apre la finestra 1024×768 con doppio buffer, profondità e stencil e accende luci, profondità e culling (`engine.cpp:255–302`); `clearWindow()` cancella colore, profondità e stencil (`373–377`); `render(camera, list, light)` disegna un frame (`385–406`); `swapBuffers()` mostra l'immagine, calcola il tempo tra due frame e gli FPS, legge gli eventi (`456–475`); `free()` chiude la finestra (`308–323`).
Altri metodi	`isRunning`, `setBackgroundColor`, `setKeyboardCallback`, `setWireframe` / `isWireframe` (scena a linee o piena, tasto F), `renderLegend` (legenda dei tasti in testo bianco in alto a sinistra, font 5×7 disegnato a quadratini, `engine.cpp:414–450`), `getDeltaTime`, `isKeyDown`, `getMouseDelta`.
Usata da	solo dal viewer: `main.cpp:186–190, 223–236`.

Object (astratta) — `object.h:14–35`, `object.cpp`

Eredita da	nessuno: è la radice della gerarchia.
Dati	`id` numero univoco, `name` nome (di solito quello scritto nel file), `idCounter` statico condiviso da tutti, parte da 1 (`object.cpp:10`) e cresce a ogni nuovo oggetto (`object.cpp:16`).
Metodi	`getId()`, `getName()`, e `render(modelView)` **virtuale puro** (`= 0`, `object.h:27`): è ciò che rende Object astratta e obbliga ogni figlia concreta a dire come si disegna. Distruttore virtuale (`object.h:20`): cancellando un oggetto tramite un puntatore alla base parte il distruttore giusto.
Copia	vietata (`object.h:19, 22`): due oggetti con lo stesso id non devono esistere.

Node — `node.h:14–36`, `node.cpp`

Eredita da	Object (id, nome, obbligo di `render`).
Dati	`matrix` matrice locale, cioè relativa al padre, inizializzata a identità (`node.cpp:14`); `parent` il padre (`nullptr` per la radice); `children` i figli.
Metodi	`setMatrix` / `getMatrix`; `getWorldMatrix()` = matrice mondo del padre × matrice locale, calcolata risalendo fino alla radice (`node.cpp:48–53`); `addChild(child)` imposta il padre del figlio e lo mette in lista (`node.cpp:59–63`); `getParent`, `getChildren`; `render` vuoto (`node.cpp:87–88`): un nodo semplice non ha niente da disegnare, serve solo a raggruppare e spostare i figli.
Distruttore	cancella tutti i figli (`node.cpp:20–24`): cancellare la radice cancella tutta la scena.
Creata da	il viewer (radice, `main.cpp:192`) e il Loader (un nodo per ogni nodo del file, `loader.cpp:144`).

Mesh — `mesh.h:14–43`, `mesh.cpp`

Eredita da	Node → quindi anche da Object. Una mesh ha una matrice, un padre e figli come ogni nodo.
Dati aggiunti	`displayList` (i triangoli già compilati), `triangleCount`, `worldMin` / `worldMax` (la scatola che la contiene, in coordinate mondo), `material` (`shared_ptr`, `mesh.h:42`).
Metodi	`build(positions, normals, uvs)` registra i triangoli in una display list e calcola la scatola (`mesh.cpp:42–70`); `setMaterial` / `getMaterial`; `getTriangleCount`; `isFlat()` = altezza della scatola sotto 1 mm, così il motore riconosce il pavimento senza conoscerne il nome (`mesh.cpp:103–106`); `getWorldMinY` / `MaxY`.
Riscrive	`render`: carica la matrice, applica il materiale, esegue la display list (`mesh.cpp:130–136`).
Particolarità	`renderGeometry()` è privata e disegna solo i triangoli senza materiale; la può chiamare soltanto `List` grazie a `friend class List` (`mesh.h:34–36`), per l'ombra. È `final`.

Camera — `camera.h:15–30`, `camera.cpp`

Eredita da	Node. Grazie a questo una camera si posiziona come qualunque altro oggetto, con una matrice.
Dati aggiunti	`fovDegrees` (angolo di vista verticale, 60°), `nearPlane` (0,05 m), `farPlane` (100 m).
Metodi	`getViewMatrix()` = inversa della matrice mondo della camera (`camera.cpp:31–34`): se la camera si sposta a destra, il mondo deve spostarsi a sinistra. `getProjectionMatrix(aspect)` = prospettiva (`camera.cpp:41–44`).
Riscrive	niente: usa il `render` vuoto di Node (una camera non si disegna).
Creata da	il viewer, 3 camere: libera, frontale, dall'alto (`main.cpp:209–215`).

Light (astratta) — `light.h:15–38`, `light.cpp`

Eredita da	Node. Una luce ha una posizione nella scena e può essere figlia di un oggetto.
Perché astratta	il costruttore è `protected` (`light.h:30`): fuori dalla gerarchia nessuno può scrivere `new Light(...)`, si deve scegliere un tipo concreto.
Dati aggiunti	`lightId` = quale delle luci di OpenGL usa (`GL_LIGHT0` + contatore, `light.cpp:21`; OpenGL 1.x ne garantisce almeno 8); `position` in coordinate omogenee: **w = 0** luce direzionale, **w = 1** luce puntiforme; `color`; `enabled`.
Metodi	`setEnabled` / `isEnabled` (tasti 1 e 2); `getWorldPosition()` = matrice mondo × posizione (`light.cpp:52–55`), serve all'ombra.
Riscrive	`render`: se spenta la disattiva, altrimenti passa posizione e colori a OpenGL (`light.cpp:61–77`).

DirectionalLight — `directionallight.h:15–21`

Eredita da	Light → Node → Object.
Cosa fa di diverso	Solo il costruttore: passa a Light la posizione `(0, 0, 1, 0)` (`directionallight.cpp:15–17`). w = 0 significa “direzione”: la luce arriva dall'asse +Z del nodo e viaggia verso −Z, la stessa direzione in cui guarda una camera.
Riscrive	niente: il `render` di Light basta.
Nel viewer	la **torcia** (`main.cpp:206`); a ogni frame riceve la matrice della camera attiva (`main.cpp:226`), quindi illumina ciò che si guarda: è la luce **dinamica**.

PointLight — `pointlight.h:14–28`, `pointlight.cpp`

Eredita da	Light → Node → Object.
Dati aggiunti	`attenuation` = (costante, lineare, quadratica); di default (1, 0, 0), cioè nessuna attenuazione (`pointlight.cpp:17–19`). Posizione passata a Light: `(0, 0, 0, 1)`, il centro del nodo.
Metodi	`setAttenuation(c, l, q)`: l'intensità diventa 1 / (c + l·d + q·d²).
Riscrive	`render`: prima chiama il `render` di Light, poi aggiunge i tre valori di attenuazione (`pointlight.cpp:42–48`). È un esempio di metodo che *estende* quello della madre invece di sostituirlo.
Nel viewer	`LampLight`, la luce della lampada, creata dal Loader leggendo il file (`loader.cpp:126–128`); è la luce **statica** e quella che proietta l'ombra.

Material — `material.h:14–39`, `material.cpp`

Eredita da	Object, **non** Node: un materiale non ha una posizione nella scena, è una proprietà di una mesh.
Dati	colori `ambient`, `diffuse`, `specular`, `emission` (tutti neri all'inizio, `material.cpp:16–20`), `shininess` (limitata a 0–128, `material.cpp:79`), `texture` (`shared_ptr`, può essere vuota).
Metodi	un `set...` per ogni dato; `getEmission()` serve al viewer per trovare il paralume e spegnerne il bagliore.
Riscrive	`render`: manda i colori a OpenGL con `glMaterial` e, se c'è, attiva la texture (`material.cpp:95–110`).

Texture — `texture.h:14–27`, `texture.cpp`

Eredita da	Object.
Dati	`textureId`, il numero che OpenGL assegna all'immagine (`texture.cpp:19–22`).
Metodi	`load(fileData, fileSize)` decodifica il PNG dalla memoria con stb_image, lo gira in verticale (OpenGL ha l'origine in basso) e lo carica nella scheda video (`texture.cpp:38–55`).
Riscrive	`render`: `glBindTexture`, cioè “usa questa immagine per i prossimi triangoli” (`texture.cpp:61–64`).

List — `list.h:15–32`, `list.cpp`

Eredita da	Object: anche una lista “si disegna”.
Dati	`lights` e `meshes`, due elenchi di puntatori (`list.h:30–31`). La lista **non possiede** i nodi: non li cancella mai.
Metodi	`pass(node)` visita l'albero e mette ogni luce in `lights` e ogni mesh in `meshes` (`list.cpp:35–44`); `clear()` svuota gli elenchi; `renderShadow(view, light)` disegna l'ombra sul pavimento (`list.cpp:84–129`).
Riscrive	`render`: **prima tutte le luci, poi tutte le mesh** (`list.cpp:60–73`). L'ordine conta: OpenGL 1.x illumina un triangolo solo con le luci già impostate quando il triangolo viene disegnato. Nello stesso giro scrive 1 nello stencil sui pixel del pavimento (`list.cpp:69`).

Loader — `loader.h:14–19`, `loader.cpp`

Eredita da	nessuno. `final`, nessun dato.
Metodi	uno solo, statico: `load(file) : Node` (`loader.cpp:178–194`). Restituisce la radice dell'albero letto, o `nullptr` se il file non si apre. Le funzioni di supporto (`readColor`, `readTexture`, `readMaterial`, `readMesh`, `readLight`, `readNode`) sono in un namespace anonimo del `.cpp`, invisibili fuori.
Usata da	`main.cpp:193`.

2.3 Le 9 frecce di ereditarietà

Tutte con il triangolo vuoto ——▷ verso la madre. In C++ ognuna corrisponde a una riga `class Figlia : public Madre`. "Pubblica" significa che tutto ciò che è pubblico nella madre resta pubblico nella figlia.

#	Freccia	Cosa riceve in eredità	Cosa aggiunge o riscrive	Dove l'eredità viene sfruttata
1	Node → Object `node.h:14`	id, nome, `getId`, `getName`, obbligo di `render`	matrice, padre, figli; `render` vuoto	Ogni nodo ha un nome stampabile: `printNode` mostra l'albero all'avvio (`main.cpp:50–61`).
2	Material → Object `material.h:14`	id, nome, obbligo di `render`	colori, lucidità, texture; `render` = `glMaterial`	`Mesh::render` chiama `material->render(...)` come per qualunque Object (`mesh.cpp:134`).
3	Texture → Object `texture.h:14`	id, nome (il percorso dell'immagine nel file)	`load`; `render` = `glBindTexture`	`Material::render` chiama `texture->render(...)` (`material.cpp:106`).
4	List → Object `list.h:15`	id, nome ("render list")	elenchi luci/mesh, `pass`, `clear`, `renderShadow`; `render` = luci poi mesh	`Base::render` chiama `list->render(view)` (`engine.cpp:404`).
5	Mesh → Node `mesh.h:14`	tutto di Node: matrice, padre, figli, `getWorldMatrix`, `addChild`	triangoli, materiale, scatola; `render` = disegna	Il Loader appende la mesh a un nodo con `addChild` (`loader.cpp:156`); `build` usa `getWorldMatrix()` ereditata (`mesh.cpp:44`).
6	Camera → Node `camera.h:15`	tutto di Node	angolo di vista, piani near/far, vista e proiezione	Le camere si mettono nell'albero (`main.cpp:214–215`) e si muovono con `setMatrix` ereditato (`main.cpp:172, 212–213`); la vista nasce da `getWorldMatrix()` (`camera.cpp:33`).

7	Light → Node `light.h:15`	tutto di Node	slot OpenGL, posizione omogenea, colore, acceso/spento; `render` = `glLight`	La luce della lampada è figlia di un nodo del file e segue la lampada (`loader.cpp:163–164`); la torcia si muove con `setMatrix` (`main.cpp:226`).
8	DirectionalLight → Light `directionallight.h:15`	tutto di Light, incluso il suo `render`	solo il costruttore (w = 0)	Il viewer salva la torcia in un `DirectionalLight*` (`main.cpp:37`) e usa `setEnabled` / `isEnabled` ereditati (`main.cpp:121`).
9	PointLight → Light `pointlight.h:14`	tutto di Light	attenuazione; `render` esteso (chiama quello di Light e aggiunge l'attenuazione)	Il viewer la tiene come `Light*` generico (`main.cpp:38`) trovato con `findLight` (`main.cpp:85–94`): non gli interessa il tipo esatto.

> **Dove si vede il polimorfismo.** `List::pass` riceve un generico `Node*` e chiede "sei una luce? sei una mesh?" con `dynamic_cast` (`list.cpp:37–40`): funziona solo perché Object ha metodi virtuali. Poi `List::render` chiama `light->render(...)` su un `Light*` (`list.cpp:63`): per la lampada parte `PointLight::render` (con l'attenuazione), per la torcia `Light::render`, perché DirectionalLight non lo riscrive. Stessa riga di codice, due comportamenti.

2.4 Le altre 6 frecce

#	Freccia	Tipo	Cosa significa e perché questo tipo	Nel codice
10	Node ◆— Node *"figli", 0..**	 composizione	Un nodo **possiede** da zero a molti figli. Freccia dalla classe a sé stessa: padre e figli sono tutti Node (o sue figlie, quindi anche Mesh, Camera, Light). Composizione perché il padre cancella i figli nel suo distruttore: i figli non sopravvivono al padre. Ogni figlio ha un solo padre.	dati `node.h:34–35`; `addChild` `node.cpp:59–63`; distruzione `node.cpp:20–24`; tutto cancellato da `delete root` (`main.cpp:235`)
11	Mesh ◇— Material	 aggregazione	Una mesh **usa** un materiale (al massimo uno; può non averlo). Aggregazione e non composizione perché il materiale è condiviso: il Loader crea ogni materiale una volta e lo dà a tutte le mesh che lo usano, tramite `shared_ptr`. Il materiale sparisce solo quando l'ultima mesh che lo usa viene cancellata.	dato `mesh.h:42`; assegnato in `loader.cpp:157`; usato in `mesh.cpp:133–134` (con il controllo "se c'è")
12	Material ◇— Texture	 aggregazione	Un materiale **usa** al massimo un'immagine. Stesso ragionamento: `shared_ptr`, la texture può essere condivisa e può mancare (pavimento e letto ce l'hanno, la lampada no).	dato `material.h:38`; creata in `loader.cpp:49–52` e assegnata in `loader.cpp:73`; usata in `material.cpp:103–109`

13	List ◇— Node *“luci e mesh”, 0..**	 aggregazione	La lista **tiene un riferimento** a molti nodi, ma non li possiede: i nodi appartengono allo scene graph. Per questo è aggregazione. In realtà tiene solo due tipi di Node: le luci e le mesh (le camere e i nodi semplici vengono saltati). Il viewer svuota la lista prima di cancellare la scena, così non restano puntatori a oggetti cancellati.	dati `list.h:30–31`; riempita da `pass` `list.cpp:35–44`; svuotata `main.cpp:234` prima di `main.cpp:235`
14	Loader ⇢ Node *“crea”*	 dipendenza	Il Loader **crea** l'albero di nodi e lo consegna a chi l'ha chiamato, che da quel momento ne è il proprietario. Dipendenza (tratteggiata) perché il Loader non conserva niente: non ha dati, i nodi non restano “suoi”. Oltre ai Node crea anche Mesh, Material, Texture e luci: il diagramma mostra solo Node perché è il tipo restituito.	firma `loader.h:18`; `new Eng::Node` `loader.cpp:144`; `new Eng::Mesh` `155`; luci `121, 126`; restituito a `main.cpp:193` e appeso a `root` in `main.cpp:200`
15	Base ⇢ List *“disegna”*	 dipendenza	Base **riceve** la lista come parametro di `render` e la disegna, ma non la conserva: la lista appartiene al viewer (variabile locale di `main`). Quindi dipendenza, non aggregazione.	firma `engine.h:82`; uso `engine.cpp:404–405`; lista creata in `main.cpp:217`

2.5 Cosa il diagramma non disegna (per restare semplice)

- **Base ⇢ Camera** e **Base ⇢ Light**: anche camera e luce sono parametri di `render`. Dalla camera Base prende proiezione e vista (`engine.cpp:395, 403`), la luce la passa a `renderShadow` (`engine.cpp:405`).
- **List → Mesh «friend»**: List può chiamare il metodo privato `Mesh::renderGeometry()` (`mesh.h:34`, usato in `list.cpp:113`).
- **List → Light, List → Mesh**: gli elenchi interni sono già divisi per tipo (`list.h:30–31`); il diagramma li riassume nella freccia “luci e mesh” verso Node.
- **Loader ⇢ Mesh, Material, Texture, DirectionalLight, PointLight**: tutti creati dal Loader (vedi freccia 14 e diagramma 5).
- **Base → Reserved**: la struttura nascosta con i dati di FreeGLUT (PIMPL, sezione 1.3); è un dettaglio interno che non fa parte dell'API.

2.6 Lo stesso metodo `render()`, comportamenti diversi

Ogni classe che deriva da Object ha `render(modelView)`. Il parametro è la matrice *model-view* = vista della camera × matrice mondo dell'oggetto: porta i vertici dalle coordinate dell'oggetto a quelle della camera.

Classe	Cosa fa il suo `render`	Dove
Object	niente: è solo dichiarato (virtuale puro)	`object.h:27`
Node	niente: un nodo semplice non ha geometria	`node.cpp:87–88`
Camera	non lo riscrive: usa quello vuoto di Node	—
Mesh	carica la matrice, chiama il `render` del materiale, esegue la display list	`mesh.cpp:130–136`
Light	se spenta, `glDisable`; altrimenti posizione, colore diffuso e speculare, niente ambiente, `glEnable`	`light.cpp:61–77`
DirectionalLight	non lo riscrive: usa quello di Light	—
PointLight	quello di Light + attenuazione costante, lineare, quadratica	`pointlight.cpp:42–48`

Material	`glMaterial` per i 4 colori e la lucidità; attiva e collega la texture, o disattiva le texture	`material.cpp:95–110`
Texture	`glBindTexture`	`texture.cpp:61–64`
List	render di tutte le luci, poi di tutte le mesh, scrivendo 1 nello stencil sul pavimento	`list.cpp:60–73`

2.7 Chi chiama chi durante un frame

Seguire una riga del `main` fino a OpenGL fa vedere tutte le frecce del diagramma al lavoro:

1. `main` chiama `eng.render(camera, &list, lamp)` (`main.cpp:229`).
2. `Base::render` imposta il viewport e la proiezione presa dalla **Camera**, calcola la vista e chiama `list->render(view)` (`engine.cpp:393–404`) — freccia 15.
3. `List::render` per ogni **Light**: `light->render(view × mondo)` — parte `PointLight::render` o `Light::render` (frecce 7, 8, 9).
4. Poi per ogni **Mesh**: `mesh->render(view × mondo)` (`list.cpp:67–71`).
5. `Mesh::render` chiama `material->render` (freccia 11), che chiama `texture->render` (freccia 12), poi `glCallList`.
6. Infine `Base::render` chiama `list->renderShadow(view, lamp)` (`engine.cpp:405`): calcola la matrice che schiaccia i mobili sul piano del pavimento vista dalla lampada (formula *MakeShadowMatrix* della SuperBible, `list.cpp:98–100`), disegna i mobili schiacciati solo nello stencil (1 → 2, `list.cpp:102–114`), poi ridisegna il pavimento *con la lampada spenta* solo dove lo stencil vale 2 (`list.cpp:119–126`). Risultato: ombra della lampada, mentre la torcia continua a illuminare il pavimento. Il paralume viene saltato perché sta all'altezza della luce (`list.cpp:110–111`).

3. Ciclo di vita del viewer

Un diagramma di **sequenza**: mostra *in che ordine* il `main()` del viewer chiama il motore, dall'avvio alla chiusura. È lo schema richiesto dalla Series 3.

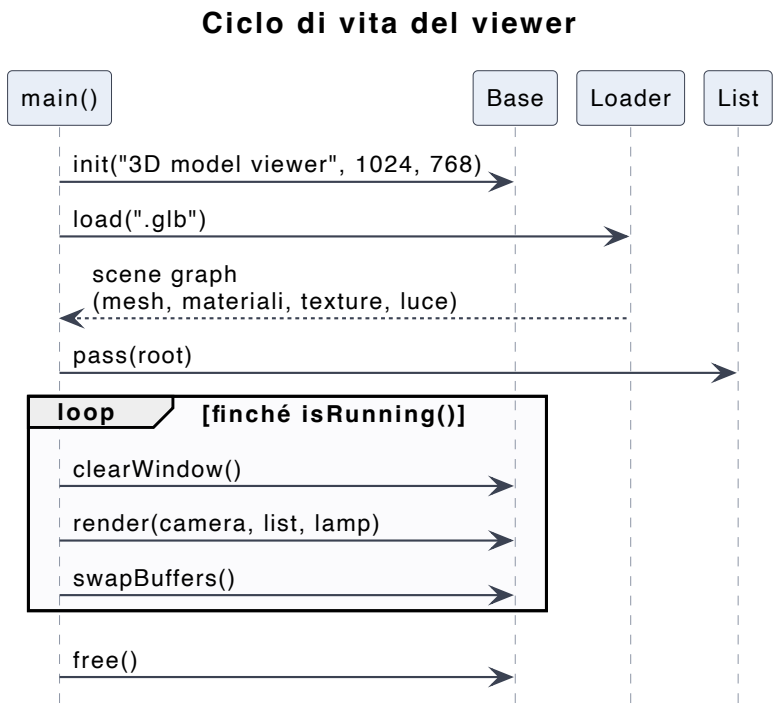


Figura 3 – Sequenza principale del viewer.

3.1 Come si legge

- Le quattro caselle in alto sono i **partecipanti**: `main()` (il viewer), `Base`, `Loader`, `List`. Sotto ognuna scende la sua linea della vita.
- Le frecce **continue** sono chiamate di metodo, nell'ordine in cui avvengono (dall'alto in basso).
- La freccia **tratteggiata** che torna verso `main()` è il valore restituito da `load`.
- Il riquadro **loop** contiene le chiamate ripetute; la condizione tra parentesi quadre, `[finché isRunning()]`, dice quando smettere.

3.2 I messaggi uno per uno

#	Freccia	Cosa succede	Nel codice
1	main → Base `init("3D model viewer", 1024, 768)`	Apre la finestra con doppio buffer (si disegna in un'immagine nascosta e la si mostra solo quando è finita), buffer di profondità e stencil; registra le funzioni per tastiera, mouse e chiusura; nasconde il cursore; accende test di profondità, eliminazione delle facce posteriori, illuminazione, normalizzazione delle normali.	`main.cpp:187` → `engine.cpp:255–302`
2	main → Loader `load(".glb")`	Legge il file e costruisce l'albero (dettagli nella sezione 6.3). Deve venire **dopo** `init`: display list e texture si possono creare solo quando esiste già il contesto OpenGL della finestra.	`main.cpp:193` → `loader.cpp:178–194`

3	Loader ⇠ main *scene graph*	Ritorna il puntatore alla radice dell'albero letto, con dentro mesh, materiali, texture e la luce. Se vale `nullptr` il viewer chiude subito (`main.cpp:194–199`). Altrimenti lo appende alla propria radice, cerca la luce della lampada e il paralume, crea la torcia e le tre camere.	`main.cpp:200–215`
4	main → List `pass(root)`	Visita tutto l'albero una volta sola e prepara gli elenchi luci e mesh. Fatto prima del ciclo, non a ogni frame, perché l'albero non cambia struttura mentre il programma gira (cambiano solo le matrici).	`main.cpp:217–218` → `list.cpp:35–44`
—	**loop** `[finché isRunning()]`	`isRunning()` diventa falso quando si preme ESC o si chiude la finestra.	`main.cpp:223`; ESC `engine.cpp:167–171`; chiusura `engine.cpp:211–214`
5	main → Base `clearWindow()`	Cancella l'immagine nascosta: colore di sfondo, profondità, stencil.	`main.cpp:228` → `engine.cpp:373–377`
6	main → Base `render(camera, list, lamp)`	Disegna la scena vista dalla camera attiva, con l'ombra della lampada (catena completa in 2.7).	`main.cpp:229` → `engine.cpp:385–406`
7	main → Base `swapBuffers()`	Mostra l'immagine finita, calcola il tempo dall'ultimo frame (serve a muoversi alla stessa velocità su PC veloci e lenti), ogni secondo scrive gli FPS nel titolo, elabora tastiera e mouse arrivati nel frattempo.	`main.cpp:231` → `engine.cpp:456–475`
8	main → Base `free()`	Fuori dal ciclo: chiude la finestra e FreeGLUT.	`main.cpp:236` → `engine.cpp:308–323`

3.3 Cosa succede anche, ma non è nel diagramma

Passo	Perché serve	Dove
`setKeyboardCallback`, `setBackgroundColor`	Dopo `init`: registra la funzione del viewer per i tasti C/1/2/F/P e imposta il grigio scuro di sfondo.	`main.cpp:189–190`
`findLight`, `findGlowingMesh`	Trovano la luce della lampada e il paralume **senza usare nomi**: la prima luce dell'albero e la prima mesh con emissione maggiore di zero.	`main.cpp:68–94, 202–205`
`printNode(root)`	Stampa nella console l'albero con la posizione mondo di ogni nodo e i triangoli di ogni mesh (14 837 in totale).	`main.cpp:221`
`moveFreeCamera`	A ogni frame, se è attiva la camera libera, legge W A S D Q E e il mouse e aggiorna la sua matrice.	`main.cpp:147–173, 225`
`torch->setMatrix(...)`	A ogni frame la torcia copia la matrice della camera attiva: è questo che la rende la luce **dinamica**.	`main.cpp:226`
`renderLegend`	Scrive in bianco la legenda dei tasti sopra la scena, in alto a sinistra, prima dello scambio dei buffer.	`main.cpp:230`
`list.clear()`, `delete root`	Svuota la lista, poi cancella la radice: per la composizione Node–Node vengono cancellati a catena tutti i 20 nodi della scena.	`main.cpp:234–235`

4. Scene graph del viewer

Un diagramma degli **oggetti**: non le classi, ma gli oggetti veri che esistono mentre il viewer gira, e chi è figlio di chi.

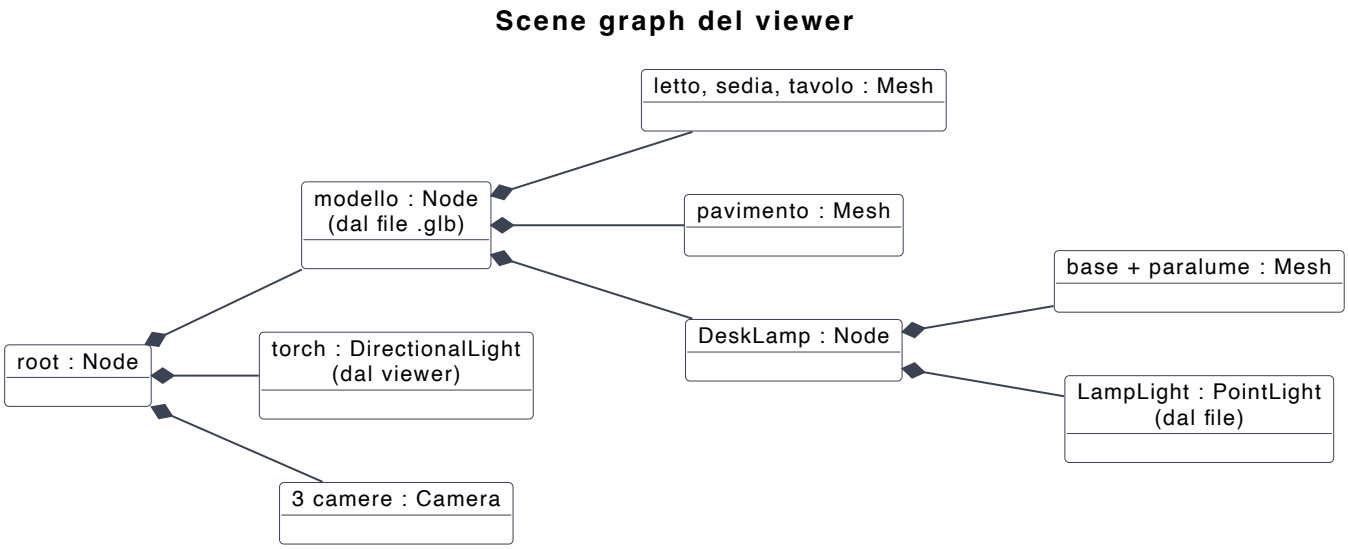


Figura 4 – Oggetti a runtime (semplificato).

4.1 Come si legge

Ogni riquadro è scritto `nome : Classe`: per esempio `torch : DirectionalLight` è l'oggetto di nome “torch” della classe DirectionalLight. Tutte le frecce sono **composizioni** (rombo pieno dal lato del padre): sono le istanze concrete della freccia 10 del diagramma 2 (*Node possiede 0..* Node*). Ogni freccia corrisponde a una chiamata `padre->addChild(figlio)`.

4.2 Gli oggetti

Oggetto	Chi lo crea e cos'è	Dove
`root : Node`	Il viewer. Radice di tutto: un nodo vuoto con matrice identità.	`main.cpp:192`
`modello : Node`	Il Loader: la radice dell'albero letto dal file. Il file ha due radici (DeskLamp e Sketchfab_model), quindi Assimp aggiunge un nodo sopra di loro.	`main.cpp:193`, `loader.cpp:193`
`letto, sedia, tavolo : Mesh`	Il Loader. Nel file letto, scrivania e sedia sono **una sola mesh** (l'autore del modello le ha unite).	`loader.cpp:155–158`
`pavimento : Mesh`	Il Loader. È l'unica mesh piatta: `isFlat()` la riconosce e riceve l'ombra.	`loader.cpp:155–158`, `mesh.cpp:103–106`
`DeskLamp : Node`	Il Loader. Nodo della lampada da scrivania, posto sul tavolo.	`loader.cpp:144`
`base + paralume : Mesh`	Il Loader. Nel file è una mesh con due materiali; Assimp la divide in **due Mesh**, una per materiale. Il paralume ha emissione (brilla).	`loader.cpp:149–159`
`LampLight : PointLight`	Il Loader, dalla luce scritta nel file (estensione `KHR_lights_punctual`): colore arancio (1; 0,55; 0,2), intensità 2,5. Luce statica.	`loader.cpp:116–130, 161–164`
`torch : DirectionalLight`	Il viewer. Colore bianco-azzurro. Luce dinamica.	`main.cpp:206–207`

`3 camere : Camera`	Il viewer: “free camera” (si muove), “front camera” in (0; 2; 4,5), “top camera” in (4; 3,5; 4), tutte rivolte al centro della scena.	`main.cpp:209–215`

4.3 Le 8 frecce

#	Freccia (padre ◆— figlio)	Cosa significa	Chiamata `addChild`
1	root ◆— modello	Tutto il modello del file si muove con root.	`main.cpp:200`
2	root ◆— torch	La torcia è figlia diretta di root, non della camera: la sua matrice viene copiata da quella della camera attiva a ogni frame, così può seguire qualunque camera.	`main.cpp:207`
3	root ◆— 3 camere	Le camere stanno nell'albero come gli altri oggetti; la vista nasce dalla loro matrice mondo. (Nel disegno un solo riquadro rappresenta tre oggetti.)	`main.cpp:214–215`
4	modello ◆— letto, sedia, tavolo	Mesh dei mobili (nel file sta alcuni livelli più in basso, vedi 4.4).	`loader.cpp:156` + `147`
5	modello ◆— pavimento	Mesh del pavimento (anche questa più in basso nel file).	`loader.cpp:156` + `147`
6	modello ◆— DeskLamp	La lampada è una delle due radici del file.	`loader.cpp:146–147`
7	DeskLamp ◆— base + paralume	Le due mesh della lampada sono figlie del suo nodo.	`loader.cpp:156`
8	DeskLamp ◆— LampLight	La luce è dentro la lampada: se si spostasse la lampada, la luce la seguirebbe. Il Loader la appende al nodo che ha lo **stesso nome** della luce.	`loader.cpp:161–164`

4.4 L'albero completo, come lo costruisce davvero il Loader

Il diagramma salta i nodi intermedi che non aggiungono significato. Questo è l'albero completo (nomi presi dal file `.glb`; in parentesi la classe del motore). Sono 20 oggetti in tutto: 11 Node, 4 Mesh, 1 PointLight, 1 DirectionalLight, 3 Camera.

```
root                                  (Node)        main.cpp:192
├── [radice aggiunta da Assimp]       (Node)        = "modello" nel diagramma
│   ├── DeskLamp                      (Node)        traslazione (0,93; 0,935; −0,846)
│   │   ├── base della lampada        (Mesh)        LampBaseMat
│   │   ├── paralume                  (Mesh)        LampShadeMat, emissione 1/0,85/0,6
│   │   └── LampLight                 (Node)        traslazione (0; 0,394; 0)
│   │       └── LampLight             (PointLight)  stesso nome del nodo
│   └── Sketchfab_model               (Node)        rotazione −90° attorno a X
│       └── 366aa845…fbx              (Node)        rotazione +90° attorno a X, scala 0,01
│           └── RootNode              (Node)
│               ├── Plane             (Node)        rotazione −90° attorno a X, scala 200
│               │   └── Plane_2_0     (Node)
│               │       └── pavimento (Mesh)        texture a griglia
│               └── SM_Bed_01         (Node)
│                   └── SM_Bed_01_1_0 (Node)
│                       └── letto + scrivania + sedia  (Mesh)  3 immagini
├── torch                             (DirectionalLight)    main.cpp:206
├── free camera                       (Camera)      main.cpp:209
├── front camera                      (Camera)      main.cpp:210
└── top camera                        (Camera)      main.cpp:211
```

Ordine dei figli: per ogni nodo il Loader aggiunge prima le sue mesh, poi le luci con lo stesso nome, poi i nodi figli (`loader.cpp:149–167`). Le due rotazioni ±90° di Sketchfab_model e del nodo .fbx vengono dall'esportazione del modello originale e si annullano a vicenda; la scala 0,01 converte i centimetri dell'originale in metri.

4.5 Esempio: dove sta la luce della lampada

La posizione nel mondo si ottiene moltiplicando le matrici lungo il percorso dalla radice (`node.cpp:48–53`). root e la radice di Assimp sono identità, quindi:

- DeskLamp: traslazione (0,93; 0,935; –0,846) → la base della lampada è sul piano del tavolo, a 93,5 cm da terra;
- nodo LampLight: traslazione locale (0; 0,394; 0) → 39,4 cm più in alto, dentro il paralume;
- PointLight: posizione locale (0; 0; 0; 1) e matrice identità.

Mondo = T(0,93; 0,935; –0,846) × T(0; 0,394; 0) = **T(0,93; 1,329; –0,846)**: la lampadina è a 1,33 m da terra. È questo il punto da cui `List::renderShadow` proietta l'ombra (`list.cpp:98`).

4.6 Dall'albero alla List

`list.pass(root)` visita l'albero in profondità e conserva solo luci e mesh (`list.cpp:35–44`):

Elenco	Contenuto, in ordine
`lights`	LampLight (PointLight), torch (DirectionalLight)
`meshes`	base della lampada, paralume, pavimento, letto + scrivania + sedia
saltati	gli 11 Node semplici e le 3 Camera (non si disegnano)

5. Il modello 3D che useremo

“Bedroom Furniture Set – Game Ready”: letto, scrivania e sedia su un pavimento. Noi abbiamo aggiunto una lampada da scrivania con il paralume luminoso e la sua luce.

Dato	Valore
Mesh	4 (letto + mobili, pavimento, base e paralume della lampada)
Triangoli	14 837, molto sotto il limite di 100k del requisito sui 30 FPS
Materiali e texture	4 materiali, 4 immagini PNG 1024×1024 dentro il file
Luci	1 luce puntiforme `LampLight` nel paralume (arancio, intensità 2,5)
Dimensioni	circa 4 × 1,3 × 4 m (scala reale, in metri)
Autore e licenza	General of Thailand (sketchfab.com/Melonpolygons), CC BY 4.0, modificato

Perché questo oggetto

- **Pavimento piatto**: superficie su cui proiettare l'ombra (requisito “ombre o riflessi su un piano”).
- **Lampada**: posto naturale per la luce statica puntiforme; la luce dinamica direzionale è la torcia che segue la camera.
- **Texture vere** (coperta, legno): si vede subito se il texture mapping funziona.
- **Pochi triangoli**: il viewer resta fluido anche su PC senza scheda video potente.

6. Il formato .glb: cos'è, perché, come lo usiamo

6.1 Cos'è un file .glb

È **glTF 2.0 in versione binaria**, uno standard aperto del **Khronos Group** (lo stesso gruppo che cura OpenGL), pensato apposta per caricare scene 3D pronte per il disegno in tempo reale. Un solo file, in tre parti:

Parte	Contenuto	Nel nostro file
Intestazione	12 byte: la parola `glTF`, la versione (2), la lunghezza totale	4,45 MB in tutto

Blocco JSON	la descrizione della scena: nodi, mesh, materiali, texture, luci	5,7 kB
Blocco BIN	i dati veri: vertici, normali, coordinate texture, immagini PNG	4,45 MB (quasi tutto immagini)

Contenuto del nostro file:

Elemento	Quanti	Dettaglio
Nodi	9	ognuno con traslazione, rotazione, scala e figli (elenco completo in 4.4)
Mesh	3 (4 dopo Assimp)	letto + mobili, pavimento, lampada (la lampada ha 2 materiali, quindi diventa 2 mesh)
Materiali	4	base della lampada, paralume (con **emissione**), pavimento, letto e mobili (con speculare)
Immagini	4 PNG	colore del pavimento; colore, normali, occlusione/rugosità del letto
Luci	1	luce puntiforme `LampLight` (estensione standard `KHR_lights_punctual`)

6.2 Perché l'abbiamo scelto

- **Contiene tutto quello che chiede il requisito** (cg_introduction, slide 33: modello da file esterno con *materiali, luci e texture*), in un solo file.
- **Un solo file = demo sicura.** Le texture sono dentro il .glb: copiando la demo su un altro PC non si perde nessuna immagine (slide 36: demo “one-click, foolproof”).
- **La gerarchia dei nodi è già uno scene graph**, come il nostro `Node`: il Loader copia l'albero così com'è.
- **Stesse convenzioni di OpenGL**: sistema destrorso, asse Y verso l'alto, unità in metri. Non serve nessuna conversione di assi o di scala.
- **Standard aperto e gratuito**: Sketchfab lo offre come download, Blender lo apre e lo salva senza plugin (il corso ammette Blender al posto di 3ds Max), Assimp lo legge.

Formato	Geometria	Materiali	Texture	Nodi	Luci	Note
.glb	**sì**	**sì**	**sì, dentro il file**	**sì**	**sì**	standard aperto, un solo file
.ply	**sì**	**no**	**no**	**no**	**no**	per scansioni 3D (punti, colori per vertice)
.obj + .mtl	**sì**	**sì**	file separati	**no**	**no**	niente nodi né matrici
.fbx	**sì**	**sì**	**sì**	**sì**	**sì**	formato proprietario Autodesk

6.3 Come lo usa il motore

`Loader::load` fa leggere il file ad Assimp con due opzioni: `aiProcess_Triangulate` (ogni poligono diventa triangoli) e `aiProcess_GenSmoothNormals` (crea le normali se mancano) (`loader.cpp:181`). Poi converte ogni parte nella classe del motore corrispondente.

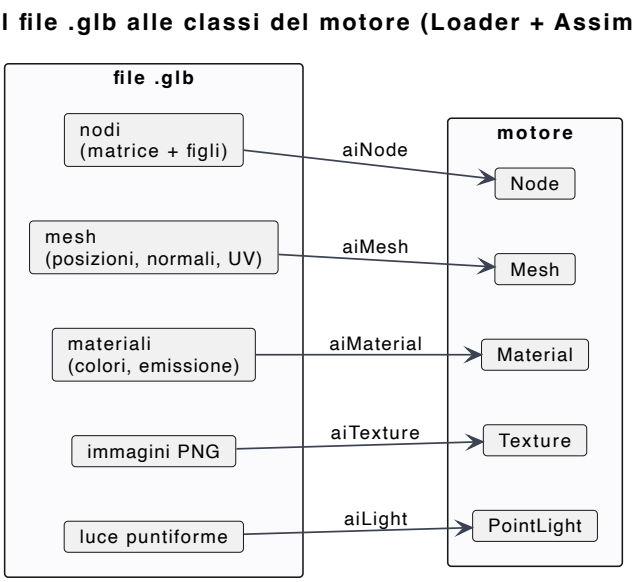


Figura 5 – Ogni parte del file diventa un oggetto del motore.

Nel diagramma i due rettangoli sono raggruppamenti (“file .glb” a sinistra, “motore” a destra). Ogni freccia continua si legge “**diventa**”; l'etichetta è la struttura intermedia di Assimp attraverso cui passa il dato.

#	Freccia	Cosa succede	Nel codice
1	nodi → Node *aiNode*	Per ogni nodo del file: `new Node` con lo stesso nome; la matrice viene **trasposta** perché Assimp la scrive per righe e GLM per colonne; il nodo viene appeso al padre; poi la funzione richiama sé stessa sui figli (ricorsione), così l'albero viene copiato identico.	`readNode` `loader.cpp:142–170`; trasposta `145`; ricorsione `166–167`
2	mesh → Mesh *aiMesh*	Per ogni mesh del nodo fatta di triangoli: `new Mesh`, appesa al nodo **prima** di costruirla (così `build` conosce già la sua matrice mondo e può calcolare la scatola in coordinate mondo); poi per ogni triangolo copia posizione, normale e UV dei 3 vertici e chiama `Mesh::build`, che li registra in una display list. Punti e linee vengono saltati.	`loader.cpp:149–159`; `readMesh` `83–108`; `Mesh::build` `mesh.cpp:42–70`
3	materiali → Material *aiMaterial*	Letti **tutti all'inizio**, prima dei nodi, perché ogni mesh indica il suo materiale con un numero (`mMaterialIndex`). Colore diffuso usato anche come ambiente, più speculare, emissione e lucidità.	ciclo `loader.cpp:190–191`; `readMaterial` `61–75`; uso dell'indice `157`
4	immagini PNG → Texture *aiTexture*	Mentre legge un materiale, il Loader cerca la sua texture di colore. Nel .glb è un PNG ancora compresso dentro il file (`mHeight == 0`, e `mWidth` è la sua lunghezza in byte); `Texture::load` lo decodifica dalla memoria.	`readTexture` `loader.cpp:39–53`; `Texture::load` `texture.cpp:38–55`
5	luce puntiforme → PointLight *aiLight*	Per ogni nodo, il Loader cerca le luci con il suo stesso nome (in glTF la luce è “attaccata” a un nodo) e crea una PointLight figlia di quel nodo, con colore (colore × intensità) e attenuazione $1/d^2$ calcolati da Assimp. Se nel file ci fosse una luce direzionale diventerebbe una DirectionalLight; gli altri tipi vengono ignorati.	ricerca `loader.cpp:161–164`; `readLight` `116–130`

Il viewer poi riconosce le parti importanti **senza nomi scritti nel codice**:

- **pavimento** = la mesh piatta (`isFlat()`): riceve l'ombra;
- **paralume** = la mesh con emissione maggiore di zero: il tasto 2 spegne insieme la luce e il suo bagliore (`main.cpp:124–134`);
- **lampada** = la prima luce trovata nel ramo caricato: è la luce statica e proietta l'ombra.

6.4 Perché è perfetto per questo progetto

Requisito del corso	Come lo soddisfa il nostro .glb
caricare un modello da un file esterno	un solo file, letto da `Loader::load`
proprietà dei materiali	4 materiali con colore, speculare ed emissione
sorgenti di luce	la luce `LampLight` arriva dal file e diventa la luce statica
texture mapping	4 immagini PNG dentro il file + coordinate UV per ogni vertice
due luci di tipo diverso, una statica e una dinamica	puntiforme dal file (statica) + direzionale del viewer (dinamica)
più di 30 FPS sotto i 100k triangoli	14 837 triangoli
ombre o riflessi su una superficie piana	pavimento piatto nel file, ombra proiettata dalla lampada
scene graph nel motore	9 nodi con matrici e figli, copiati uno a uno

Appendice A – Riepilogo di tutte le frecce

Diagramma	Frecce	Tipi usati	Spiegate in
1. Architettura	8	6 continue (uso diretto), 2 tratteggiate (dipendenza da GLM)	1.2
2. Classi	15	9 generalizzazioni, 1 composizione, 3 aggregazioni, 2 dipendenze	2.3, 2.4
3. Ciclo di vita	8 + 1	8 chiamate, 1 valore restituito, 1 riquadro loop	3.2
4. Scene graph	8	8 composizioni (una per `addChild`)	4.3
5. Caricamento .glb	5	5 continue (“diventa”)	6.3

Appendice B – I file del progetto

File	Righe	Contiene
`engine/engine.h`, `engine.cpp`	98 + 505	Base (finestra, input, render, legenda), macro `ENG_API`, include di tutte le classi
`engine/object.h`, `.cpp`	37 + 41	Object
`engine/node.h`, `.cpp`	38 + 88	Node
`engine/mesh.h`, `.cpp`	45 + 144	Mesh
`engine/camera.h`, `.cpp`	32 + 44	Camera
`engine/light.h`, `.cpp`	40 + 77	Light
`engine/directionallight.h`, `.cpp`	23 + 23	DirectionalLight
`engine/pointlight.h`, `.cpp`	30 + 48	PointLight
`engine/material.h`, `.cpp`	41 + 110	Material
`engine/texture.h`, `.cpp`	29 + 64	Texture (con stb_image)
`engine/list.h`, `.cpp`	34 + 129	List e ombra
`engine/loader.h`, `.cpp`	21 + 194	Loader (Assimp)
`client/main.cpp`	240	il viewer
Totale	**2 199**	commenti Doxygen inclusi

`doc/uml/*.puml`	—	sorgenti PlantUML dei 5 diagrammi (le figure sono i `.svg` generati)